

CAPÍTULO 2

CADENAS DE CARACTERES Y ESTRUCTURA DE DATOS



2.1. Lección 4

Durante esta lección aprenderemos a manipular cadenas de caracteres. Desarrollaremos funciones que nos permitirán realizar operaciones básicas, tales como: averiguar la longitud de una cadena, obtener una subcadena, convertir de mayúsculas a minúsculas, averiguar en qué posición aparece un determinado carácter, etcétera. Todas estas funciones integrarán nuestra propia biblioteca y formarán la base sobre la cual, más adelante, construiremos otras bibliotecas de funciones más avanzadas.

2.1.1. Carácter

Llamamos así a cualquiera de los símbolos que componen un alfabeto. Por ejemplo: la letra 'A', el número '5' o el signo '+'.


Cada carácter se identifica con un valor entero positivo. Esta relación carácter/valor es fija y se establece en una tabla conocida como *Tabla ASCII*, que veremos a continuación.

Por ejemplo, al carácter 'A' le corresponde el valor 65, al carácter

Lección 4

'B' el valor 66, y así sucesivamente. Además, la tabla ASCII asigna el valor 48 al carácter '0'; 49 al carácter '1'; 50 al carácter '2', etcétera.

32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
p	q	r	s	t	u	v	w	x	y	z	{		}	~	

Tabla 2.1. Tabla ASCII

En C++ los caracteres se representan con el tipo de dato `char`, preparado especialmente para contener un valor entero de 1 byte de longitud: un código ASCII.

Por lo anterior, las siguientes líneas de código son equivalentes:

```
char c1='A'; | char c2=65;
```

También es válido hacer:

```
int i='A';
```

Si mostramos por consola los valores de las variables `c1`, `c2` e `i` tendremos la siguiente salida:

```
char c1='A';
char c2=65;
int i='A';

cout << c1 << endl; // Salida: A
cout << c2 << endl; // Salida: A
cout << i << endl;   // Salida: 65
cout << (char)i << endl; // Salida: A
```

2.1.2. Cadena de caracteres

Una cadena de caracteres es una sucesión finita y ordenada de caracteres. “Hola”, “Pablo” y “Esto es una cadena” son cadenas de caracteres. Si cambiamos el orden de los caracteres de cualquiera de estas cadenas, ya no serán las mismas, serán otras.

Aunque C++ no provee un tipo de dato primitivo que permita contener cadenas de caracteres, existe la clase `string` que cubre ese vacío.

2.1.2.1. Caracteres especiales y carácter de escape

A priori una cadena no podría incluir el carácter " (comilla doble), porque el mismo se utiliza para indicar dónde comienza y finaliza dicha cadena de caracteres.

```
string s = "El caracter " es un problema"; // ERROR
```

El carácter " (comilla doble) se considera especial, pero esto no es un impedimento para que una cadena pueda contener caracteres especiales. Simplemente habrá que anteponerles el *carácter de escape* \ (carácter contrabarra).

```
string s = "El caracter \" ya no es un problema";
// SALIDA: El caracter " ya no es un problema
cout << s << endl;
```

Al anteponer \ al carácter " (comilla doble), el compilador interpretará que " es parte de la cadena; no su delimitador.

Otros caracteres especiales son:

Carácter especial	Descripción
\n	Salto de línea
\r	Retorno de carro
\t	Tabulador
\"	Comilla doble
\\	Contrabarra
\0	Carácter nulo

Tabla 2.2. Caracteres especiales

Otro ejemplo de uso de un carácter especial podría ser:

```
string s = "El caracter\nes un salto de linea";
cout << s << endl;
```

La salida será:

```
El caracter
es un salto de linea
```

2.1.2.2. Carácter nulo que indica el final de una cadena

Cuando asignamos: `string s="Hola"`, internamente se genera una estructura como la siguiente:

H	o	l	a	\0
0	1	2	3	4

Figura 2.1. Estructura de una cadena de caracteres

Los caracteres de la cadena se alojan en celdas de memoria consecutivas que se numeran desde cero. Al final, se agrega una celda adicional que contiene el carácter especial ‘\0’ (barra cero), también llamado carácter nulo, que permite identificar dónde finaliza el contenido de la cadena de caracteres.

Es muy importante tener presente que todas las cadenas finalizan con ‘\0’, pues este concepto será fundamental para desarrollar las funciones de tratamiento de cadenas de caracteres que plantearemos más adelante. De hecho, como podemos ver en el siguiente ejemplo, `cout` reconoce al ‘\0’ como fin de la cadena.

```
string s1="Hola, esta es \0mi cadena";
cout << s1 << endl; // SALIDA: Hola, esta es
```

2.1.2.3. Acceso directo a los caracteres de una cadena

Podemos acceder a cada uno de los caracteres de una cadena `s` haciendo referencia a `s[i]`, donde `i` es un valor comprendido entre 0 y la posición del carácter nulo.

Por ejemplo:

```
string s = "Hola";

cout << s[0] << endl; // Salida: H
cout << s[1] << endl; // Salida: o
cout << s[2] << endl; // Salida: l
cout << s[3] << endl; // Salida: a
cout << s[4] << endl; // Salida: [sin salida, caracter nulo]
```

NOTA: Dependiendo de qué versión del compilador estemos usando, el código anterior podría no compilar, e indicar el siguiente error de compilación:

```
Invalid overload of 'endl'
```

Si fuera el caso, lo solucionaremos casteando `s[i]` a `char`.

```
string s = "Hola";

cout << (char)s[0] << endl; // Salida: H
// :
```

2.1.2.4. Longitud de una cadena

La longitud indica cuántos caracteres válidos contiene una cadena de caracteres. Por ejemplo, la cadena “Hola” tiene una longitud igual a 4; a su vez, este valor coincide con la posición que ocupa el carácter nulo.

2.1.2.5. Operadores aritméticos unarios

Este tipo de operadores son alternativos a los operadores aritméticos tradicionales y simplifican notablemente su uso. En la siguiente tabla, podemos ver a los operadores unarios más frecuentes, y también compararlos con los operadores aritméticos tradicionales equivalentes.

Sean las variables `i`, `a` y `b` de tipo `int` y con algún valor válido, entonces:

Tradicional	Unario	Descripción
<code>i=i+1</code>	<code>i++</code>	Incrementa el valor de <code>i</code> en una unidad.
<code>i=i-1</code>	<code>i--</code>	Decrementa el valor de <code>i</code> en una unidad.
<code>a=a+b</code>	<code>a+=b</code>	Acumula el valor de <code>b</code> en <code>a</code> .
<code>a=a-b</code>	<code>a-=b</code>	Le resta a <code>a</code> el valor de <code>b</code> .
<code>a=a*b</code>	<code>a*=b</code>	Asigna a <code>a</code> su valor multiplicado por <code>b</code> .

Tabla 2.3. Operadores aritméticos unarios

2.1.2.6. Ciclo iterativo for

El ciclo `for` es una alternativa al ciclo `while`, y consiste en una estructura de repetición que permite controlar exactamente cuántas iteraciones queremos realizar. Por ejemplo, el programa del apartado anterior puede resolverse con un ciclo `for`, tal como aparece en el código de la izquierda. A la derecha, observamos el mismo programa, pero usando un ciclo `while`.

```
string s = "Hola";
for(int i=0; i<4; i++)
{
    cout << s[i] << endl;
}
```

```
string s = "Hola";
int i=0;
while( i<4 )
{
    cout << s[i] << endl;
    i++;
}
```

El encabezado del ciclo `for` tiene tres secciones, que se separan entre sí mediante un `;` (punto y coma).

En la primera sección, se declara una variable y su valor inicial. En este ejemplo, declaramos la variable `i` cuyo valor inicial es 0 (cero). En la tercera sección, se establece un patrón de modificación de la variable. En nuestro caso, `i++` indica que la variable `i` se incrementará automáticamente una unidad por cada iteración del ciclo. Finalmente, en la sección del medio se establece una condición lógica; el ciclo `for` iterará mientras se verifique que dicha condición sea verdadera. En nuestro ejemplo, el ciclo iterará mientras que el valor de `i` sea menor que 4.

Sabiendo que las cadenas finalizan con '\0', podemos usar un ciclo `for` para iterar a través de los caracteres de la cadena `s` mientras que `s[i]` no sea el carácter nulo, es decir, mientras no lleguemos al final de la cadena.

```
string s = "Hola";
for(int i=0; s[i]!='\0'; i++)
{
    cout << s[i] << endl;
}
```

Esta solución es mejor que la anterior, ya que no depende de la longitud `s`. En otras palabras: el `for` itera y muestra todos los caracteres de la cadena, independientemente de cuál sea su longitud.

2.1.2.7. Concatenar cadenas de caracteres

Para concatenar cadenas, usamos el operador `+`. Veamos el siguiente ejemplo:

```
string s1 = "Hola";
string s2 = ", que tal?";
string s3 = s1 + s2;

cout << s3 << endl; // Salida: Hola, que tal?
```

2.1.2.8. Operadores relacionales aplicados a cadenas

Los operadores relacionales `<`, `<=`, `>`, `=>` e `==` permiten comparar alfabéticamente cadenas de caracteres. Por ejemplo:

```
string s1 = "Pablo";
string s2 = "Alberto";
string s3 = "Pablo";

if( s2<s1 ) // Salida: Alberto es alfabeticamente...
{
    cout<< s2 <<" es alfabeticamente menor que: "<< s1 <<endl;
}

if( s1==s3 ) // Salida: Pablo es igual a Pablo
{
}
```

```
cout<< s1 <<" es igual a: "<< s3 <<endl;
}
```

2.1.2.9. Función de comparación

Cuando hablamos de *función de comparación* nos referimos a una función cuyo objetivo es comparar dos elementos, que podrían o no ser del mismo tipo, a fin de establecer un criterio de precedencia. Dicho de otro modo, dados dos elementos *a* y *b*, la función de comparación debe determinar si *a precede a b*, *b precede a a*, o ambos valores son iguales.

Al existir tres posibilidades, una función de comparación que compare dos elementos *a* y *b*, debe retornar un valor entero negativo, positivo o nulo (cero), según la siguiente tabla.

Si considera que	Retorna
<i>a</i> precede a <i>b</i>	Negativo
<i>b</i> precede a <i>a</i>	Positivo
<i>a</i> es igual a <i>b</i>	0 (cero)

Tabla 2.4. Valor de retorno de una función de comparación

Por ejemplo, la siguiente función de comparación compara dos valores enteros *a* y *b* determinando que *a* precede a *b* si *a* es menor que *b*.

```
int cmpInt(int a,int b)
{
    int ret;
    if( a<b )    // si a precede a b retorno negativo
    {
        ret=-1;
    }
    else
    {
        if( a>b ) // si a no precede a b retorno positivo
        {
            ret=1;
        }
        else
        {

```

```

        ret=0;  // si a es igual a b retorno cero
    }
}

return ret;
}

```

Esta misma función podemos resolverla más fácilmente así:

```

int cmpInt(int a,int b)
{
    return a-b;
}

```

Puede sorprendernos que la implementación de la función sea tan corta, pero cumple absolutamente con todos los requisitos planteados: si $a < b$, la diferencia $a-b$ será negativa y la función retornará un valor menor que cero. Si $a > b$ la diferencia $a-b$ será positiva, y si ambos valores son iguales la diferencia será nula.

`cmpInt` establece que a precede a b si a es numéricamente menor que b . Si quisiéramos cambiar el criterio de precedencia para indicar que a precede a b si a es numéricamente mayor que b escribiremos la siguiente función:

```

int cmpInt2(int a,int b)
{
    return b-a;
}

```

Veamos una función de comparación que compara dos cadenas, determinando que $s1$ precede a $s2$ si $s1$ es alfabéticamente menor que $s2$.

```

int cmpString(string s1,string s2)
{
    int ret;

```

```

    if( s1<s2 ) // si s1<s2 retorno negativo
    {
        ret=-1;
    }
    else
    {
        if( s1>s2 ) // si s1>s2 retorno positivo
        {
            ret=1;
        }
        else
        {
            ret=0; // si s1==s2 retorno cero
        }
    }

    return ret;
}

```

Ahora veremos una función de comparación que compara elementos de diferentes tipos, como ser: una persona (Persona) y un número de DNI (int).

```

struct Persona
{
    int dni;
    string nom;
};

int cmpPersonaDNI(Persona p,int dni)
{
    return p.dni-dni;
}

```

2.1.2.10. *If-inline*

Esta sentencia es una abreviatura del `if`. En ocasiones resulta muy cómoda, y dependiendo del caso, nos permitirá ahorrar muchas líneas de código.

En el código de la izquierda, podemos ver la función de comparación de cadenas de caracteres `cmpString` que desarrollamos antes. A la derecha, vemos la misma función implementada con un *if-inline*.

<pre>int cmpString(string s1 ,string s2) { int ret; if(s1<s2) { ret=-1; } else { if(s1>s2) { ret=1; } else { ret=0; } } return ret; }</pre>	<pre>int cmpString(string s1,string s2) { // CORTO, pero... ¿¿QUE HACE?? return s1<s2?-1:s1>s2?1:0; }</pre>
---	---

Aunque pueda parecernos algo confuso, el *if-inline* tiene su lógica, y es muy simple. La estructura del *if-inline* es la siguiente:

condición ? *retorno por true* : *retorno por false*

La sentencia se compone de tres partes, separadas entre sí por ? (signo de interrogación) y : (dos puntos).

La primera parte, ubicada entre el inicio de la sentencia y el signo ?, es la expresión lógica cuyo valor de verdad debe ser evaluado. A continuación, entre el signo ? y : colocamos la segunda parte de la sentencia: qué valor queremos retornar en caso de que la expresión lógica resulte verdadera. Finalmente, la tercera parte

del *if-inline* se ubica después de `:` y consiste en el valor que la sentencia debe retornar en caso de que la expresión lógica resulte falsa.

La función `cmpString` que desarrollamos en la tabla anterior utiliza dos sentencias *if-inline* anidadas.

```
int cmpString(string s1, string s2)
{
    return s1<s2?-1:s1>s2?1:0;
}
```

Si se verifica que `s1<s2` retornamos -1. Si la expresión anterior no es verdadera entramos en el segundo *if-inline*, evaluando si `s1>s2`. En este caso retornamos 1; si no, retornamos 0. Según cuál sea el caso, la función retornará un valor negativo, cero o positivo.

2.1.3. Biblioteca de funciones y API

Una *biblioteca de funciones* consiste en un conjunto de funciones, desarrolladas por nosotros mismos o por terceros, con la finalidad de resolver cuestiones vinculadas a una problemática común. Las bibliotecas no sólo incluyen funciones, también pueden incluir TAD y clases.

Por ejemplo, una biblioteca de comunicaciones consistiría en un conjunto de funciones que podríamos invocar para hacer que nuestro programa se conecte con otros procesos a través de la red, una biblioteca de gráficos de video nos ayudaría a escribir programas que dibujen gráficos en la pantalla, y una de tratamiento de cadenas de caracteres agruparía funciones para manipular los caracteres de una cadena.

Aunque el concepto es un poco más extenso, solemos llamar API a la documentación de las funciones de una biblioteca. Esta documentación debe describir:

- Nombre de la función.
- Tipo de dato del valor de retorno.
- Parámetros que recibe.
- Tipo de dato de los parámetros.
- Qué hace.
- Qué retorna.
- Ejemplo de uso.

2.1.3.1. Tratamiento de cadenas de caracteres

Llamamos *tratamiento de cadenas de caracteres* o, simplemente, *tratamiento de cadenas* al conjunto de acciones que realizamos sobre una cadena con el objetivo de manipular sus caracteres.

Por ejemplo: dada una cadena, averiguar su longitud, si contiene o no un determinado carácter, si está en mayúsculas o minúsculas, convertirla a mayúsculas o minúsculas, remover los espacios en blanco que pueda tener a la izquierda, derecha o en ambos extremos, extraer una subcadena, etcétera.

Una biblioteca de funciones para el tratamiento de cadenas de caracteres agrupará todas estas funciones, y su API las documentará.

Invito al lector a *googlear* las palabras: *API String Java*. Con esto podrá acceder fácilmente a la documentación de todas las funciones de tratamiento cadenas que provee dicho lenguaje de programación.

A continuación, como actividad práctica, desarrollaremos nuestra propia biblioteca para el tratamiento de cadenas de caracteres, cuyas funciones tendrán nombres y prototipos casi idénticos a los de Java. Pues, como autor considero que luego de aprender a diseñar algoritmos y programarlos, Java debe ser el siguiente paso.

2.1.3.2. Actividad práctica: API de tratamiento de cadenas de caracteres



Para este desarrollo, trabajaremos sobre un proyecto Eclipse, previamente configurado con los prototipos de cada una de las funciones de la API.

A través de los siguientes códigos QR podremos descargar la API (documentación), el proyecto sobre el cual vamos a trabajar, y ver un video que explica cómo importar el proyecto en Eclipse y comenzar a desarrollar nuestro trabajo.



API de tratamiento de
cadenas de caracteres



Descargar proyecto
Eclipse



Importar el proyecto y
modalidad de trabajo

Esta modalidad de trabajo la repetiremos a lo largo de muchos de los capítulos de este curso de algoritmos y programación.

A modo de aventón, analizaremos cómo encarar y resolver las primeras dos funciones de la API. Estas funciones son: `length` e `charCount`.

La función `length` (longitud) debe retornar la longitud de una cadena que recibe como parámetro. Su prototipo es:

```
int length(string s);
```

Un ejemplo de uso de esta función podría ser el siguiente:

```
string s = "Hola";
int x = length(s);
cout << "La longitud de: " << s << " es: " << x << endl;
```

La lógica para programar `length` consiste en incrementar un contador (`i`), cuyo valor inicial será 0, el cual utilizaremos para acceder a cada uno de los caracteres de `s`; nos detendremos cuando `s[i]` sea el carácter nulo `'\0'`. En este momento el valor de `i` coincidirá con la longitud de la cadena.

```
int length(string s)
{
    int i=0;
    while( s[i]!='\0' )
    {
        i++;
    }

    return i;
}
```

La función `charCount`, que recibe una cadena `s` y un carácter `c`, debe contar cuántas veces aparece `c` dentro de `s`. Su prototipo es:

```
int charCount(string s,int c);
```

Un ejemplo de uso sería el siguiente:

```
char c = 'a';
string s = "La casa es linda";
int x = charCount(s,c);

cout << "En:" << s << ", el carácter:" << c << " ";
cout << "aparece:" << x << " veces "<< endl;
```

La lógica para resolver esta función consiste recorrer la cadena `s` preguntando por cada carácter si es igual a `c`. En tal caso incrementaremos un contador (`cont`) cuyo valor final coincidirá con la cantidad de veces que `c` aparece dentro de `s`.

```
int charCount(string s,char c)
{
    int cont = 0;
    for(int i=0; s[i]!='\0'; i++)
    {
        if( s[i]==c )
        {
            cont++;
        }
    }

    return cont;
}
```

2.1.4. Argumentos en línea de comandos

Cuando ejecutamos un programa también podemos pasarle argumentos. Se trata simplemente de cadenas de caracteres que el programa puede leer y utilizar.

Por ejemplo, en la siguiente línea, ejecutamos un programa y le pasamos 4 argumentos.

```
C:\>MuestraArgumentos Pablo Pedro Juan Raul
```



Argumentos en línea de comandos

Acceder a estos valores requiere usar recursos que todavía no hemos explicado. Aun así, esto no será un impedimento para comprender cómo hacerlo.

```
// se agregan estos dos parametros al prototipo de main
int main(int argc, char** argv)
{
    // argc indica cuantos argumentos recibe el programa
    for(int i=1; i<argc; i++)
    {
        // aunque cada argumento esta en argv[i], son string
        string s = argv[i];

        // usamos el i-esimo argumento
        cout << s << endl;
    }

    return 0;
}
```

Comenzamos agregando dos parámetros al prototipo de la función `main`, que por convención llamaremos: `argc` (*argument count*) y `argv` (*argument vector*).

El primero indica cuántos argumentos recibimos; el segundo contiene los argumentos. Recordemos que cada argumento es una cadena de caracteres. En nuestro ejemplo, cuando ejecutamos el programa le pasamos 4 argumentos; sin embargo, el programa recibirá 5 cadenas, numeradas de 0 a 4.

La primera cadena, a la que podemos acceder a través de `argv[0]`, contendrá el nombre del programa: “MuestraArgumentos”. Luego, `argv[1]` será “Pablo”, `argv[2]` “Pedro”, `argv[3]` “Juan” y `argv[4]` “Raul”.

2.1.5. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

2.2. Lección 5

En esta lección, utilizaremos las funciones que construimos durante la lección anterior para crear una nueva biblioteca de funciones de tratamiento de *tokens*. Esto es: manipular subcadenas, dentro de una cadena principal, separadas entre sí por un carácter separador. A la cadena principal la llamaremos *cadena tokenizada*.

A su vez, estas funciones serán la base sobre la cual, más adelante, construiremos una herramienta que nos permitirá administrar colecciones de objetos.

2.2.1. Tratamiento de *tokens*

Sea s una cadena y c un carácter, llamaremos *token* a toda subcadena de s encerrada entre dos caracteres c . O bien, entre el inicio de s y la primera ocurrencia de c o la última ocurrencia de c y el final de la cadena s .

Por ejemplo, si c es el carácter `'|'` (léase carácter *pipe*), y s la cadena `"John|Paul|George|Ringo"`, los *tokens* que podremos extraer de la cadena s serán: `"John"`, `"Paul"`, `"George"` y `"Ringo"`, en ese orden. Pero si el carácter c fuese `'o'`, considerando la misma cadena s , los *tokens* por extraer serán: `"J"`, `"hn|Paul|Ge"`, `"rge|Ring"` y `""`; este último *token* consiste en la cadena vacía.

Diremos que s es la cadena *tokenizada* y c el carácter separador.

Existen dos casos particulares:

- La cadena vacía no genera *tokens*, sea cual fuere el carácter separador.
- Si s no contiene a c , toda la cadena, en sí misma, será un (único) *token*.

2.2.2. Actividad práctica: API de tratamiento de *tokens*



Del mismo modo que lo hicimos con la biblioteca de cadenas de caracteres, aquí desarrollaremos una biblioteca cuyas funciones nos permitirán manipular los *tokens* que existen dentro de una cadena *tokenizada*. Por ejemplo: determinar cuántos *tokens* tiene una cadena, agregar, insertar y remover *tokens* y establecer si, dentro de la cadena, existe o no un *token* especificado.



Lección 5



API de tratamiento de *tokens*

Para programar las funciones de esta API utilizaremos la biblioteca de funciones de tratamiento de cadenas de caracteres que desarrollamos más arriba.

A modo de aventón, analizaremos cómo desarrollar las dos primeras funciones: `tokenCount` y `addToken`.

La función `tokenCount`, que recibe una cadena *tokenizada* `s` y un carácter separador `c`, debe contar y retornar cuántos *tokens* genera `c` dentro de `s`.

```
int tokenCount(string s, char c);
```

Un ejemplo de uso podría ser el siguiente:

```
// caso 1
string s = "John|Paul|George|Ringo";
int n = tokenCount(s, '|');
cout << n << endl; // Retorna 4

// caso 2
s = "Paul";
int n = tokenCount(s, '|');
cout << n << endl; // Retorna 1

// caso 3
string s = "";
int n = tokenCount(s, '|');
cout << n << endl; // Retorna 0
```

La lógica para resolver esta función requiere contar cuántas veces aparece el carácter separador `c` dentro de la cadena *tokenizada* `s`, y sumarle 1. Para eso podremos utilizar la función `charCount` de la biblioteca de cadenas de caracteres.

En la cadena `s` del ejemplo anterior, el carácter separador `'|'` aparece 3 veces, generando 4 *tokens*: “John”, “Paul”, “George” y “Ringo”.

Esta misma lógica también aplica en el caso en que la cadena `s` no contenga ningún carácter separador. En tal situación `charCount` retornará 0, y al sumarle 1 estaremos retornando 1, acorde a lo esperado; pues, si la cadena no contiene al carácter separador toda la cadena será un (único) *token* en sí misma.

Sin embargo, si se tratase de la cadena vacía la función `tokenCount` debería retornar 0, ya que la cadena vacía no contiene *tokens*. Esta situación la contemplaremos especialmente con un `if`.

El código de la función será el siguiente.

```
int tokenCount(string s, char c)
{
    return isEmpty(s)?0:charCount(s,c)+1;
}
```

Como podemos apreciar, resolvimos la función `tokenCount` en una única línea de código gracias a que, previamente, desarrollamos `isEmpty` y `charCount`.

Esta es la esencia de la programación: construir capas de software apoyándonos en capas que construimos previamente, las cuales se apoyan sobre otras construcciones anteriores y así sucesivamente.

Analicemos ahora la función `addToken`, que recibe una cadena *tokenizada* `s`, un carácter separador `c`, y una cadena `t` (*token*) que debemos agregar al final de `s`.

Su prototipo es: `void addToken(string& s, char c, string t);`

Un ejemplo de uso podría ser:

```
string s = "John|Paul|George|Ringo";
char c = '|';
string t = "George Martin";
addToken(s,c,t);

// Salida: John|Paul|George|Ringo|George Martin
cout << s << endl;

s = "";
addToken(s,c,t);

// Salida: George Martin
cout << s << endl;
```

La lógica para implementar `addToken` es tan simple como concatenar, al final de `s`, un carácter `c` seguido del *token* `t`. Pero debemos contemplar el caso particular de que `s` sea la cadena vacía. En tal situación, `t` pasará a ser `s`.

```
void addToken(string& s, char c, string t)
{
    s = isEmpty(s)?t:s+c+t;
}
```

2.2.3. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

2.3. Lección 6

A medida que adquirimos experiencia en programación tendemos a maximizar el nivel de abstracción del código que programamos, pues nos damos cuenta de que no tiene sentido repetir varias veces el mismo proceso.

Cuando una función recibe parámetros de un determinado tipo de dato, ésta sólo podrá ser usada si le pasamos argumentos del tipo en cuestión. Por ejemplo, la función `sumar`, cuyo prototipo vemos a continuación, únicamente tiene utilidad para sumar números enteros, retornando un valor del mismo tipo.

```
int sumar(int a, int b);
```

¿Qué sucedería si tuviésemos que sumar valores tipo `double`, esperando obtener como resultado un valor de este tipo de dato? ¿Y si tuviésemos que sumar fracciones (variables tipo `Fraccion`)?

Si pudiésemos abstraernos y llamar T al tipo de dato de los elementos que vamos a sumar, el prototipo de la función podría plantearse del siguiente modo:

```
T sumar(T a, T b);
```

Disponemos del operador $+$ para sumar valores numéricos, ya sean tipo `int` o `double`. Pero ¿cómo podríamos hacer que la función `sumar` sume fracciones?

Aquí veremos recursos que nos permitirán lograr tal nivel de abstracción, que será imprescindible a lo largo de todo el curso de algoritmos y programación.

2.3.1. Funciones como argumentos de otras funciones

Una función f puede recibir entre sus parámetros a otra función g , e invocarla sin conocer cuál función es ni qué hace. Cada vez que invoquemos a f , podremos pasarle una u otra función como argumento en el lugar de la función g .



Lección 6

Por ejemplo, la función `invocar`, cuyo código vemos a continuación, recibe como parámetro a la función x , la invoca pasándole valores entre 1 y 5, y se abstrae de cuál es y qué hace dicha función.

La función x podría ser cualquiera, siempre y cuando retorne un valor tipo `double` y reciba como parámetro otro valor tipo `int`.

```
void invocar(string mssg, double x(int))
{
    for(int i=1; i<5; i++)
    {
        // invocamos a la funcion f
        double d = x(i);
        cout<< mssg << " de " << i << " = " << d << endl;
    }
}
```

¿Qué hace la función `invocar`? Realmente no podemos saberlo, pues sólo invoca 5 veces a x ; mostrando el valor que retorna $x(i)$, acompañado de un mensaje que tampoco sabemos qué dice.

Enseguida veremos una función `main` que invoca a la función `invocar`, pero para esto debe pasarle, entre sus argumentos, un mensaje y una función que retorne un valor tipo `double` y reciba uno tipo `int`.

Las funciones `fact` y `fib`, que respectivamente calculan el factorial y el n -ésimo término de la sucesión de Fibonacci, realizan tareas diferentes. Sin embargo, tienen prototipos idénticos, que además concuerdan con el requisito anterior. Retornan un valor tipo `double` y esperan recibir uno tipo `int`.

```
// factorial de n
double fact(int n)
{
    double f=1;
    for(int i=2;i<=n;i++)
    {
        f=f*i;
    }

    return f;
}
```

```
// fibonacci de n
double fib(int n)
{
    double ret;
    if( n>=2 )
    {
        ret =fib(n-1)+fib(n-2);
    }
    else
    {
        ret = 1;
    }

    return ret;
}
```

Cualquiera de estas funciones podría pasar como argumento en la función `invocar`. Justamente, esto es lo que hacemos en la función `main`. Donde invocamos a `invocar` pasándole `fact`, y luego `fib`. Acompañando cada invocación con un mensaje acorde.

```
int main()
{
    // invocamos a invocar pasandole (como parametro) a fact
    invocar("factorial", fact);

    cout << " --- :O) --- " << endl; // carita sonriente

    // invocamos a invocar pasandole (como parametro) a fib
    invocar("fibonacci", fib);
}
```

```

    return 0;
}

```

La salida de este programa será:

```

factorial de 1 = 1
factorial de 2 = 2
factorial de 3 = 6
factorial de 4 = 24
--- :O) ---
fibonacci de 1 = 1
fibonacci de 2 = 2
fibonacci de 3 = 3
fibonacci de 4 = 5

```

Para comprender el alcance y potencialidad que tiene este recurso debemos estudiar otro tema fundamental: *tipos genéricos* o *templates*. Combinando ambos recursos, lograremos un altísimo nivel de abstracción, que nos permitirá construir bibliotecas genéricas y maximizar el nivel de reutilización de código.

2.3.2. Tipo de dato genérico (*template*)

Como ya sabemos, las funciones pueden recibir valores como parámetros, incluso, vimos que pueden recibir funciones entre sus parámetros. Pues bien, aquí agregaremos que una función puede parametrizar los tipos de dato de sus parámetros. Por ejemplo: la función `f` recibe un parámetro tipo `T` (tipo genérico). Luego, al invocar dicha función especificaremos concretamente cuál es ese tipo de dato.

Veamos el siguiente ejemplo, cuya explicación daremos a continuación.

<pre> //funcion f template<typename T> void f(T t) { // ... } </pre>	<pre> // invocamos a la funcion f int main() { f<int>(5); // T es int f<string>("Hola"); // T es string return 0; } </pre>
--	---

En la función `main` del recuadro de la derecha, invocamos dos veces a `f` especificando entre `<` (menor) y `>` (mayor) cuál es el tipo de dato concreto que debe tomar el parámetro `T`.

En la primera invocación, indicamos que `T` es `int` y le pasamos como argumento un valor entero (5), coincidiendo con el tipo concreto de `T`. En la segunda invocación, indicamos que `T` es `string` y le pasamos una cadena de caracteres.

Llamamos *template* a una función que parametriza los tipos de sus parámetros.

Otro ejemplo: la función `comparaYMuestra`, cuyo código veremos enseguida, compara y muestra (de menor a mayor) tres valores tipo `T`. No conoce cuál es ese tipo de dato y, en consecuencia, no sabe cómo comparar valores de ese tipo.

Sin embargo, gracias a que entre sus parámetros recibe la función de comparación `cmpTT`, que compara dos valores tipo `T`, puede abstraerse del problema y realizar su tarea sin inconvenientes.

```
template<typename T>
void comparaYMuestra(T a,T b,T c,int cmpTT(T,T))
{
    T menor,medio,mayor;

    // verifico si a es el mayor de todos
    if( cmpTT(a,b)>0 && cmpTT(a,c)>0 )
    {
        mayor = a;

        // cual esta en el medio y cual es el menor?
        if( cmpTT(b,c)>0 )
        {
            medio = b;
            menor = c;
        }
        else
        {
            medio = c;
            menor = b;
        }
    }
    else
    {
```

```

// verifico si b es el mayor de todos
if( cmpTT(b,a)>0 && cmpTT(b,c)>0 )
{
    mayor = b;

    // cual esta en el medio y cual es el menor?
    if( cmpTT(a,c)>0 )
    {
        medio = a;
        menor = c;
    }
    else
    {
        medio = c;
        menor = a;
    }
}
else
{
    // por descarte, el mayor es c
    mayor = c;

    // cual esta en el medio y cual es el menor?
    if( cmpTT(a,b)>0 )
    {
        medio = a;
        menor = b;
    }
    else
    {
        medio = b;
        menor = a;
    }
}

cout << "Menor: " << menor;
cout << "Medio: " << medio;
cout << "Mayor: " << mayor;
}

```

Para invocar a `comparaYMuestra` debemos disponer de alguna función de comparación, y pasársela como argumento en el lugar del parámetro `cmpTT`.

Anteriormente, desarrollamos las funciones `cmpInt` y `cmpString` que permiten, respectivamente, comparar enteros y cadenas. Agregaremos una tercera función para comparar cadenas según su longitud, es decir: la cadena más corta precede a la cadena más larga.

```
int cmpString2(string a,string b)
{
    return length(a)-length(b);
}
```

En el siguiente programa, invocamos a `comparaYMuestra` para mostrar el menor, medio y mayor de tres valores tipo `int`, y otros tres tipo `string`. El criterio de precedencia que determinará cuál de éstos es el menor, medio y mayor lo determina la función de comparación que le pasemos como argumento.

```
int main()
{
    int i=30,j=10,k=49;

    // Salida: mayor=10, medio=30, menor=49
    comparaYMuestra<int>(i,j,k,cmpInt);

    string s="Alberto",t="Juan",z="Pablo";

    // Salida: mayor=Alberto, medio=Juan, menor=Pablo
    comparaYMuestra<string>(s,t,z,cmpString);

    // Salida: mayor=Alberto, medio=Pablo, menor=Juan
    comparaYMuestra<string>(s,t,z,cmpString2);

    return 0;
}
```

¿Podremos usar `comparaYMuestra` para comparar estructuras y establecer algún criterio de precedencia? Por supuesto que sí.

Veamos la estructura `Persona` y dos funciones que nos permitirán comparar personas: `cmpPersonasDNI`, que compara dos personas de acuerdo con su DNI y `cmpPersonasNombre`, que lo hace considerando sus nombres.

```
struct Persona
{
    int dni;
    string nombre;
};
```

```
// compara personas por su DNI
int cmpPersonasDNI(Persona p1,Persona p2)
{
    return p1.dni-p2.dni;
}
```

```
// compara una personas por su nombre
int cmpPersonasNombre(Persona p1,Persona p2)
{
    return p1.nombre<p2.nombre?-1:p1.nombre>p2.nombre?1:0;
}
```

Esta última función podríamos simplificarla invocando a `cmpString`.

```
// compara una personas por su nombre
int cmpPersonasNombre(Persona p1,Persona p2)
{
    return cmpString(p1.nombre,p2.nombre);
}
```

Ahora sí, en el programa principal, invocamos a `comparaYMuestra` para comparar y mostrar personas según los criterios que establecen las funciones de comparación `cmpPersonasDNI` y `cmpPersonasNombre`.

```

int main()
{
    Persona p1 = {10,"Pablo"};
    Persona p2 = {20,"Alberto"};
    Persona p3 = {30,"Juan"};

    // muestra, de mayor a menor por DNI
    comparaYMuestra<Persona>(p1,p2,p3,cmpPersonasDNI);

    // muestra, de mayor a menor por nombre
    comparaYMuestra<Persona>(p1,p2,p3,cmpPersonasNombre);

    return 0;
}

```

Al compilar este programa, que conceptualmente está bien, obtendremos un error de compilación que hasta el momento no había ocurrido. El problema está relacionado con los `cout` del final de `comparaYMuestra`:

```

// :
cout<< "Menor: " << menor;
cout<< "Medio: " << medio;
cout<< "Mayor: " << mayor;
}

```

Como ahora `T` es `Persona`, `cout` *no sabe* o *no conoce* cuál es el criterio que debe utilizar para mostrar una persona en la consola. Este problema no apareció cuando `T` era `int` o `string`.

Si queremos que `comparaYMuestra` también funcione con tipos de dato definidos por el programador, debemos modificar su prototipo para que reciba una función que, dado un valor tipo `T`, retorne una cadena de caracteres que lo represente. Por convención, llamaremos a esta función `tToString`, ya que “convierte” un valor tipo `T` en un `string`.

```

template<typename T>
void comparaYMuestra(T a,T b,T c,int cmpTT(T,T)
                        ,string tToString(T))
{
    // ...
}

```

De este modo, para mostrar el menor, medio y mayor valor entre *a*, *b* y *c* debemos invocar a `tToString` a fin de obtener la cadena que representa a cada uno de estos valores e imprimirlas en la pantalla.

```
// :
cout << "Menor: " << tToString(menor);
cout << "Medio: " << tToString(medio);
cout << "Mayor: " << tToString(mayor);
}
```

2.3.3. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

2.4. Lección 7

Durante esta lección, desarrollaremos una herramienta que nos permitirá administrar colecciones de objetos. Por ejemplo: una colección de enteros, cadenas, estructuras, etcétera. Al tratarse de una colección de objetos (no importa de qué tipo) tendremos que hacer uso intensivo de los recursos que estudiamos durante la lección anterior: tipos genéricos y punteros a funciones (recibir funciones como parámetros).

La colección de objetos estará implementada sobre una cadena *tokenizada*. Para encapsular u ocultar esta cuestión, diseñaremos un TAD que llamaremos: `Coll`.

El TAD `Coll` proveerá funciones para agregar elementos a la colección, modificar o eliminar elementos ya existentes, buscar, ordenar, etcétera.

2.4.1. Colecciones

En programación, una colección consiste en un conjunto de elementos, todos del mismo tipo, agrupados bajo un identificador común; es decir, una única variable que permite hacer referencia a todos los elementos del conjunto.

Las colecciones deben proveer funciones que brinden acceso a cada uno de sus elementos, como así también funciones para ordenarlos, iterarlos, modificarlos, agregar y quitar elementos, etcétera.

Podríamos decir que `string` es una colección de caracteres, pues una única variable de este tipo agrupa e identifica a un conjunto de caracteres, y provee acceso a cada uno de sus elementos a través del operador `[]` (operador corchete).

Siguiendo el mismo razonamiento, una cadena *tokenizada* es una colección de cadenas, donde cada *token* es un elemento de la colección. En este caso, fue nuestra responsabilidad programar las funciones para implementar los métodos de acceso y demás funcionalidades.

El acceso a los elementos de una colección debe ser homogéneo. Por ejemplo: podemos utilizar un ciclo `for` para iterar sobre todos los caracteres de una cadena y abstraernos de cuál es el carácter al que estamos accediendo. Lo mismo sucede cuando accedemos a los *tokens* de una cadena *tokenizada*.

Es decir, accedemos al *i*-ésimo carácter o al *i*-ésimo *token* y le aplicamos un proceso sin preocuparnos por cuál es ese elemento; y, por lo general, todos los elementos de la colección son sometidos al mismo tratamiento.

Para comprender de qué estamos hablando, en el siguiente ejemplo, a la izquierda, todos los caracteres de la cadena `s` reciben el mismo proceso: se imprimen por consola. Análogamente, a la derecha, se calcula la longitud de cada uno de los *tokens* de la cadena *tokenizada* `x`, se acumula dicho valor y se muestra por consola la longitud promedio de los *tokens*.

```
string s="Mi cadena";
```

```
string x="Pablo|Juan|Pedro";
```



Lección 7

<pre>// cantidad de caracteres int n=length(s); // el for for(int i=0;i<n;i++) { // proceso el i-esimo // caracter cout << s[i] <<endl; }</pre>	<pre>// cantidad de tokens int n=tokenCount(s,' '); int sum=0; for(int i=0;i<n;i++) { // proceso el i-esimo token string tk=getTokenAt(x,i,' '); sum+=length(tk); } cout << sum/n <<endl;</pre>
---	--

Los lenguajes de programación ofrecen una amplia variedad de posibilidades para utilizar colecciones. Los *arrays*, por ejemplo, son un tipo primitivo de colección que estudiaremos más adelante.

Sin embargo, y dado que nuestro principal objetivo es aprender a programar, utilizaremos todos los recursos que hemos estudiado para implementar nuestra propia colección de objetos. En otras palabras: crearemos un tipo de dato que nos permitirá administrar conjuntos de objetos en memoria. Lo llamaremos: TAD Coll.

2.4.2. TAD Coll

Hasta el momento, la única posibilidad que tenemos de mantener una colección de valores en memoria es mediante una cadena *tokenizada*, y acceder a cada uno de sus elementos a través de las funciones de tratamiento de *tokens* que desarrollamos previamente, tal como vemos en el siguiente ejemplo donde implementamos, sobre dos cadenas *tokenizadas*, una colección de nombres (cadenas) y otra de “números enteros”.

```
// coleccion de nombres (cadenas)
string nombres="Juan|Pedro|Pablo";
for(int i=0; i<tokenCount(nombres);i++)
{
    string nom = getTokenAt(s,'|',i);
```

```

    cout << nom << endl;
}

// coleccion de numeros (¿numeros?)
string numeros="1|4|8|9|15";
for(int i=0; i<tokenCount(numeros);i++)
{
    string sNum = getTokenAt(s,'|',i);
    int num = stringToInt(sNum); // convertimos
    cout << num << endl;
}

```

Aunque cumple con su objetivo, trabajar con *tokens* resulta muy engorroso. Peor aún si necesitamos trabajar con una colección de valores numéricos, lo que requiere convertir entre cadena y número; o si la colección fuera de estructuras, como podría ser una colección de fechas, personas o cualquier otra.

El TAD `Coll`, que analizaremos y desarrollaremos en este apartado, nos ayudará a proveer una capa de abstracción sobre las cadenas *tokenizadas*, aportando la ilusión de disponer de una *colección de objetos*, cualquiera sea su tipo de dato.

En otras palabras, desarrollaremos un TAD a través del cual podremos administrar colecciones de elementos. Su implementación será mediante *tokens* y cadenas *tokenizadas*, ya que por el momento sólo conocemos estos recursos.

2.4.2.1. Ejemplo de uso

Para entender de qué estamos hablando, primero analizaremos un programa que utiliza dos colecciones: una de números enteros y otra de cadenas de caracteres, tal cual el programa anterior. Esto, suponiendo y aceptando que el TAD `Coll` ya está programado y disponible para ser utilizado.

```

// coleccion de numeros
Coll<int> c1 = coll<int>();
collAdd<int>(c1,1,intToString);

```

```

collAdd<int>(c1, 4, intToString);
collAdd<int>(c1, 8, intToString);
collAdd<int>(c1, 9, intToString);
collAdd<int>(c1, 15, intToString);

collReset<int>(c1);
while( collHasNext<int>(c1) )
{
    int num = collNext<int>(c1, stringToInt);
    cout << num << endl;
}

// coleccion de nombres (cadenas)
Coll<string> c2 = coll<string>();
collAdd<string>(c2, "Juan", stringToString);
collAdd<string>(c2, "Pedro", stringToString);
collAdd<string>(c2, "Pablo", stringToString);

collReset<string>(c2);
while( collHasNext<string>(c2) )
{
    string nom = collNext<string>(c2, stringToString);
    cout << nom << endl;
}

```

Como vemos, en este programa no existen *tokens* ni cadenas *tokenizadas*. En su lugar, están las colecciones `c1` y `c2`, preparadas respectivamente para contener elementos tipo `int` y `string`.

Casi todas las funciones del TAD `Coll` que utilizamos en este ejemplo reciben como parámetro una función que llamaremos: *función de conversión*.

En el caso de `collAdd`, para agregar valores enteros a `c1` le pasamos la función `intToString`, que dado un número entero retorna una cadena de caracteres que lo representa. Por otro lado, cuando iteramos sobre la colección `c1`, le pasamos a `collNext` la función inversa: `stringToInt`, que dada una cadena con un valor numérico retorna ese mismo valor como `int`. Ambas funciones las hemos desarrollado como parte de la API de tratamiento de cadenas de caracteres.

Análogamente, para agregar valores a `c2` e iterar dicha colección de cadenas, le pasamos como argumento a `collAdd` y `collNext` la función `stringToString` que ya desarrollamos, cuya funcionalidad es trivial: retorna la misma cadena que recibe.

```
string stringToString(string s)
{
    return s;
}
```

El porqué de estas necesidades lo descubriremos cuando llegue el momento de programar las funciones del TAD `Coll`. Podemos deducir que, dado que la implementación del TAD se basa en *tokens* y cadenas *tokenizadas*, cada elemento que se agregue a la colección será un *token* dentro de una cadena.

2.4.2.2. Estructura del TAD `Coll`

El TAD `Coll` utilizará la siguiente estructura, y su implementación quedará a cargo del estudiante.

```
template<typename T>
struct Coll
{
    // implementacion a cargo del estudiante
};
```

2.4.2.3. Actividad práctica: API del TAD `Coll`



Como ya mencionamos, la implementación de las funciones que integran la API del TAD será responsabilidad del estudiante. Sin embargo, tal como lo hicimos con las API de cadenas y *tokens*, aquí analizaremos y resolveremos dos de estas funciones, y parte de la estructura `Coll`.

Comenzaremos con la estructura `Coll`, cuyo objetivo es ocultar (encapsular) la implementación de la colección de objetos; que, como ya sabemos, utilizará una cadena *tokenizada* para guardar cada uno de los elementos de la colección.



API del TAD `Coll`

```
template<typename T>
struct Coll
{
    // cadena tokenizada
    string s;
    char sep;

    // :
};
```

Las funciones que analizaremos son: `coll` (función de inicialización) y `collAdd`, que agrega un elemento al final de la colección de objetos.

La función `coll` tiene por objetivo inicializar una colección. Esto es: preparar los campos de la estructura `Coll` para que, a través de las funciones de la API, el usuario pueda agregar elementos y manipularlos. Su prototipo es:

```
Coll<T> coll();
```

El siguiente ejemplo ilustra el uso de las funciones `coll` y `collAdd`. Esta última la analizaremos posteriormente.

```
Coll<int> c = coll<int>();
collAdd<int>(c, 5, intToString);
collAdd<int>(c, 11, intToString);
collAdd<int>(c, 13, intToString);

// :
```

Dado que el objetivo de la función `coll` es crear una estructura `Coll`, dar valor inicial sus campos y retornarla, su implementación es sencilla; casi trivial.

```
template<typename T>
Coll<T> coll()
{
```

```

Coll<T> c;
c.s = "";
c.sep = '|';
return c;
}

```

Observemos que en esta implementación de `coll` *hardcodeamos* el carácter separador. Pero la API incluye una versión sobrecargada de `coll` que permite especificar qué valor debe tomar dicho carácter.

A través de la función `collAdd` podremos agregar un elemento al final de la colección. Su prototipo es:

```
int collAdd(Coll<T>& c, T t, string tToString(T));
```

La función debe retornar la posición en la cual quedó colocado el elemento que acabamos de agregar.

El ejemplo de uso de esta función es el mismo que utilizamos para `coll`.

La implementación es muy sencilla, pues requiere sólo dos líneas.

```

template<typename T>
int collAdd(Coll<T>& c, T t, string tToString(T))
{
    addToken(c.s, c.sep, tToString(t));
    return tokenCount(c.s, c.sep) - 1;
}

```

Como podemos apreciar, la implementación de las funciones de `Coll` es muy simple gracias a que disponemos de API de tratamiento de *tokens*, cuyas implementaciones también resultaron fáciles de resolver por disponer de la API de tratamiento de cadenas de caracteres.

2.4.3. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

2.5. Lección 8

Frecuentemente necesitaremos ordenar los elementos de una colección, o buscar dentro de ésta la existencia de algún elemento que concuerde con ciertas características.

El TAD `Coll` tiene en cuenta estos requerimientos y ofrece las funciones `collSort` y `collFind` que, respectivamente, permiten ordenar la colección y buscar elemento determinado. En este apartado, estudiaremos algunos algoritmos con los cuales podremos implementar las funciones mencionadas.

2.5.1. Ordenamiento

En todos los casos que aquí analizaremos trabajaremos sobre una colección de enteros `Coll<int>`, y el criterio de prece-
dencia será de menor a mayor. Porque nuestro interés principal, aquí y ahora, es comprender la lógica de los algoritmos, no así su generalización.



Lección 8

2.5.1.1. Ordenamiento por inserción simple

Este algoritmo es extremadamente fácil de implementar. Requiere crear una colección auxiliar, inicialmente vacía, que llamaremos `aux`, y llamaremos `c` a la colección que vamos a ordenar.

El algoritmo consiste en buscar el menor elemento de `c`, removerlo y agregarlo al final de `aux`. Repetiremos este proceso hasta que la colección `c` quede vacía.

Finalmente, la colección `aux` tendrá todos los elementos de `c` ordenados ascendentemente. El último paso será asignar `aux` a `c`.

```
void insetionSort(Coll<int>& c)
{
    // coleccion auxiliar vacia
    Coll<int> aux = coll<int>();

    // itero mientras queden elementos en c
    while( collSize<int>(c)>0 )
    {
        // busco el menor en c
        int pos = 0;
        int min = collGetAt<int>(c,0,stringToInt);
        for(int i=0; i<collSize<int>(c);i++)
        {
            int elm = collGetAt<int>(c,i,stringToInt);
            if( elm<min )
            {
                min = elm;
                pos = i;
            }
        }

        collAdd<int>(aux,min,intToString);
        collRemoveAt<int>(c,pos);
    }

    c = aux;
}
```

2.5.1.2. Ordenamiento por burbujeo

Este es el algoritmo que habitualmente se utiliza para explicar cómo ordenar una colección. Consiste en recorrer la colección tomando sus elementos *de a pares*, y si corresponde ordenarlos entre sí.

Tomamos el primer elemento y lo comparamos con el segundo. Si están desordenados (es decir, el segundo es menor que el primero) los permutamos. Luego, tomamos el segundo y lo comparamos con el tercero, y así sucesivamente.

Este proceso colocará al mayor elemento de la colección en el último lugar, ocupando así su lugar definitivo. Después de repetir el proceso tantas veces como elementos tenga la colección, ésta quedará ordenada.

```
void bubbleSort(Coll<int>& c)
{
    int n = collSize<int>(c);
    for(int i=0; i<n; i++)
    {
        for(int j=0; j<n-1; j++)
        {
            int a = collGetAt<int>(c, j, stringToInt);
            int b = collGetAt<int>(c, j+1, stringToInt);

            if( a>b )
            {
                int aux = a;
                collSetAt<int>(c, b, j, intToString);
                collSetAt<int>(c, aux, j+1, intToString);
            }
        }
    }
}
```

2.5.1.3. Ordenamiento por burbujeo mejorado

El burbujeo tradicional repite el proceso de comparación n veces, donde n es el tamaño de la colección que ordenaremos. Sin embargo, si la colección está parcialmente ordenada es probable que no sea necesario repetirlo tantas veces.

La mejora consiste en verificar si durante la iteración actual se realizó, al menos, una permutación. Si dicha iteración concluyó y no fue necesario permutar ningún elemento será porque todos los elementos de la colección ya están en orden, señal de que podemos detener el proceso.

```
void bubbleSortMejorado(Coll<int>& c)
{
    int n = collSize<int>(c);
    bool ordenado = false;

    while(!ordenado)
    {
```

```

ordenado = true;
for(int j=0; j<n-1;j++)
{
    int a = collGetAt<int>(c,j,stringToInt);
    int b = collGetAt<int>(c,j+1,stringToInt);

    if( a>b )
    {
        int aux = a;
        collSetAt<int>(c,b,j,intToString);
        collSetAt<int>(c,aux,j+1,intToString);
        ordenado = false;
    }
}
}
}

```

2.5.1.4. Ordenamiento por inserción avanzado

Este método de ordenamiento es similar al de inserción simple, pero no requiere una colección auxiliar. Utiliza para tal fin una parte de la misma colección.

Llamaremos T a toda colección. Dentro de T , una parte será C (la colección que vamos a ordenar) y otra será A (el espacio auxiliar).

Necesitaremos una variable p para delimitar, dentro de T , hasta dónde llega C y dónde comienza A .

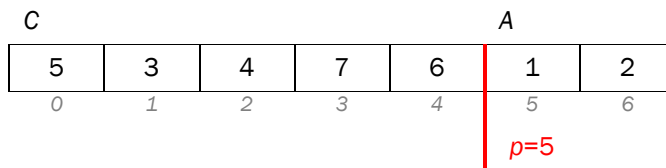


Figura 2.2. Colección separada en dos partes: real y auxiliar

Como vemos en la imagen, tenemos una única colección T que representa dos colecciones: C y A , donde C abarca los elementos ubicados entre las posiciones 0 y $p-1$, y A los elementos que se ubican a partir de p .

Al inicio, la colección A estará vacía, por lo que todos los elementos de T serán la colección C . El valor de p coincidirá con el tamaño de la colección. Luego, buscamos el menor elemento en C (entre 0 y p), lo removemos y lo agregamos al

final de A. Al remover un elemento de C, los elementos de A se desplazarán hacia la izquierda, por lo que tendremos que decrementar el valor de p . Cuando p llegue a 0, C habrá quedado vacía y A tendrá los elementos de C ordenados.

```
void insetionSortAdv(Coll<int>& c)
{
    int p = collSize<int>(c);

    while( p>0 )
    {
        int pos = 0;
        int min = collGetAt<int>(c,0,stringToInt);
        for(int i=0; i<p; i++)
        {
            int elm = collGetAt<int>(c,i,stringToInt);
            if( elm<min )
            {
                min = elm;
                pos = i;
            }
        }

        collAdd<int>(c,min,intToString);
        collRemoveAt<int>(c,pos);
        p--;
    }
}
```

2.5.2. Búsqueda

2.5.2.1. Búsqueda lineal

La *búsqueda lineal* es la primera solución que pensamos cuando hablamos de buscar un elemento dentro de una colección, porque además de ser intuitiva es fácil de implementar. Consiste en recorrer los elementos de la colección, del primero al último, y detener el proceso cuando encontramos lo que buscamos. La función retorna la posición en la cual encontró lo que buscaba o negativo si no lo encontró.

```
int search(Coll<int> c,int v)
{
```

```

int i = 0;

// recorro mientras no encuentre y no llegue el final
int n = collSize<int>(c);
while(i<n && collGetAt<int>(c,i,stringToInt)!=v )
{
    i++;
}

// retorno la posicion donde encuentre negativo
return i<n?i:-1;
}

```

La búsqueda lineal recorre la colección buscando un elemento. Si el elemento no existe o se ubica entre los últimos lugares de la colección, el algoritmo deberá evaluar toda la colección para dar una respuesta.

2.5.2.2. Búsqueda binaria

La *búsqueda binaria*, también llamada *dicotómica*, divide la colección en mitades. Cada mitad la vuelve a dividir, una y otra vez, hasta dar con el elemento buscado, o determinar que dicho elemento no existe dentro de la colección.

Este algoritmo es mucho más eficiente que el anterior, pero como contrapartida requiere que los elementos del conjunto estén ordenados.

Veamos una colección de números enteros, ordenada de menor a mayor, donde buscaremos el valor 28 para determinar si lo contiene, y en qué posición.

El algoritmo utiliza un índice i para señalar el primer elemento de la colección, un índice j para señalar el último, y calcula la posición intermedia $k = (i+j)/2$.

3	6	8	14	17	23	28	32	39	51
0	1	2	3	4	5	6	7	8	9
i				k					j

Debemos comparar el elemento que encontramos en la posición k de la colección con el valor que estamos buscando. Los llamaremos respectivamente e y v .

Si $e < v$, entonces, de existir v en la colección, seguro estará ubicado en la mitad de la derecha, porque al estar ordenada, todos los elementos ubicados a la izquierda serán más pequeños aún.

Para descartar la mitad izquierda de la colección movemos el índice i a la posición $k+1$ y volvemos a calcular k como el promedio entre i y j .

3	6	8	14	17	23	28	32	39	51
0	1	2	3	4	5	6	7	8	9
					i		k		j

En este caso, resulta que $e > v$ (32 es mayor que 28), entonces, de existir v se ubicará en la mitad izquierda de lo que queda de la colección. Para descartar la mitad derecha movemos j a $k-1$ y volvemos a calcular k .

3	6	8	14	17	23	28	32	39	51
0	1	2	3	4	5	6	7	8	9
					i	k	j		

El elemento que encontramos (23) es un valor menor que el que buscamos (28), así que debemos mover i a la derecha de k .

3	6	8	14	17	23	28	32	39	51
0	1	2	3	4	5	6	7	8	9
						i	j	k	

Al final, encontramos lo que buscamos en la posición 6, promedio entre i y j .

De no existir el elemento buscado, los índices i y j terminarán cruzados ($i > j$), y esto nos dará la pauta de que la búsqueda finalizó.

```
int binarySearch(Coll<int> c,int v)
{
    int k;
    bool enc = false;
    int i = 0;
    int j = collSize<int>(c)-1;

    while( i<=j && !enc)
    {
        k = (i+j)/2;
        int elm = collGetAt<int>(c,k,stringToInt);
```

```

    if( elm<v )
    {
        i=k+1;
    }
    else
    {
        if( elm>v )
        {
            j=k-1;
        }
        else
        {
            enc=true;
        }
    }
}

return i<=j?k:-1;
}

```

2.5.3. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

2.6. Lección 9

Durante esta lección estudiaremos *estructuras de datos*. Entendiendo el concepto de *estructura* como *conjunto de datos*, y *estructura de datos* como una construcción o anidamiento de conjuntos de datos.

Los contenidos que aquí trataremos parten de suponer y aceptar que el lector desarrolló y probó todas y cada una de las funciones de la API del TAD `Coll`,

presentado y discutido en los apartados anteriores. De no ser así, sugiero detener la lectura y completar dicha tarea antes de continuar con la presente lección.

2.6.1. Estructura de datos (parte 1)

Una *estructura de datos* es un conjunto de datos. Cuando declaramos un tipo de dato usando la palabra `struct` creamos una estructura compuesta por varios datos, tantos como campos agrupa dicha estructura. La colección de objetos que implementamos con el TAD `Coll` permite agrupar un conjunto de datos; por lo tanto, es una estructura de datos.



Lección 9

2.6.1.1. Estructuras estáticas y dinámicas

Esta clasificación se refiere a la posibilidad de agregar, modificar o eliminar datos sobre la estructura, y al hecho de que cada una de estas acciones se corresponda con un mayor o menor consumo de memoria. Es decir: que pueda crecer o decrecer.

Las estructuras que declaramos con `struct` son estáticas. Esto significa que la cantidad de datos que agrupa una estructura no puede ser modificada durante la ejecución del programa, y el único modo de alterar dicha cantidad es modificando el código fuente y volviendo a compilar.

La estructura `X`, cuyo código vemos a la derecha, declara un conjunto capaz de contener 5 valores enteros, e insume 20 bytes de memoria, ya que cada `int` ocupa 4 bytes. Alterar dicha capacidad, agregando, quitando o modificando campos, requiere modificar y recompilar su código fuente.

```
struct X
{
    int c1;
    int c2;
    int c3;
    int c4;
    int c5;
};
```

Las variables tipo `X` siempre ocuparán la misma cantidad de memoria, más allá de que usemos o no todos sus campos.

En el siguiente código, `x1` y `x2` (tipo `X`) ocupan la misma cantidad de memoria.

```
X x1; // uso solo 2 campos, igual insume 20 bytes
```

```

x1.c1 = 1;
x1.c2 = 2;

X x2; // uso todos sus campos, igual insume 20 bytes
x2.c1 = 1;
x2.c2 = 2;
x2.c3 = 3;
x2.c4 = 4;
x2.c5 = 5;

```

Por su parte, el TAD `Coll` implementa una estructura de datos dinámica. Esto significa que la cantidad de elementos del conjunto puede variar durante la ejecución del programa; y por supuesto, la colección insumirá más memoria a medida que le vayamos agregando más elementos.

2.6.1.2. Estructura de datos como cimiento del algoritmo

La estructura de datos es el soporte del algoritmo. Una estructura mal diseñada complica la solución, haciendo que el algoritmo se vuelva excesivamente complejo o inviable. En cambio, una estructura adecuada asegura un algoritmo fluido, simple y eficaz.

Por el momento, sólo contamos con los recursos `struct` y `Coll`, suficiente para comenzar a tejer nuevas estructuras que resulten de la combinación de ambos. Por ejemplo: una colección de colecciones, una colección de estructuras, una colección de estructuras donde cada una contiene una colección, etcétera.

Desde ahora, no sólo pensaremos en diseñar un algoritmo. También será nuestra tarea pensar y construir una estructura de datos que le de soporte.

2.6.1.3. Colección de estructuras

Analizaremos un ejemplo que muestra cómo crear una colección de estructuras. Comencemos declarando la estructura `Persona`.

```

struct Persona
{
    int dni;

```

```
string nom;
};
```

Al declarar: `Coll<Persona> c`, la variable `c` nos da acceso a un conjunto de datos que, gráficamente, podríamos representar del siguiente modo:

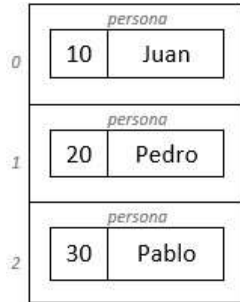


Figura 2.3. Colección de estructuras

Por la naturaleza del TAD `Coll`, internamente cada elemento de la colección es un *token* dentro de una cadena *tokenizada*. En este caso, dicho *token* es una cadena que representa una persona.

Para operar con una colección de personas, primero debemos disponer de las funciones de conversión y comparación.

La conversión entre `Persona` y `string`, y viceversa la llevan a cabo las funciones de conversión: `personaToString` y `personaFromString`, necesarias para invocar a `collAdd`, `collSetAt`, `collNext` y `collGetAt`, entre otras.

```
string personaToString(Persona p)
{
    return intToString(p.dni)+" "+p.nombre;
}

Persona personaFromString(string s)
{
    Persona p;
    p.dni = stringToInt(getTokenAt(s,0,', '));
    p.nom = getTokenAt(s,1,', ');
}
```

```

    return p;
}

```

Es muy importante observar que la cadena que diseñamos para representar a una persona usa el carácter `,` (coma) para separar entre sí los atributos (o campos) de la persona que representa. Si tenemos tres personas: `{10, "Juan"}`, `{20, "Pedro"}` y `{30, "Pablo"}`, sus cadenas de representación serán: `"10,Juan"`, `"20,Pedro"` y `"30,Pablo"`. Al agregarlas a una colección, su cadena *tokenizada* (separada con `'|'`) quedará del siguiente modo:

```
"10,Juan|20,Pedro|30,Pablo"
```

Vemos que la colección de estructuras no es más que una simple cadena *tokenizada*. En otras palabras, una colección de estructuras encapsula una cadena *tokenizada* donde cada *token* es, en sí mismo, una cadena *tokenizada*.

Ahora, desarrollaremos dos funciones de comparación: `cmpPersonaDNI`, que compara una persona (su DNI) con un número de DNI; y `cmpPersonaNombre`, que realiza la comparación entre una persona (su nombre) y un nombre.

Estas funciones serán necesarias si quisiéramos invocar a `collFind` para buscar, dentro de la colección, una persona que tenga un determinado número de DNI o un cierto nombre.

```

int cmpPersonaDNI(Persona p,int dni)
{
    return p.dni-dni;
}

int cmpPersonaNombre(Persona p,string nom)
{
    return p.nom<nom?-1:p.nom>nom?1:0;
}

```

Finalmente, para invocar a `collSort` desarrollaremos tres funciones que nos permitirán comparar personas: `cmpPersonasNomAsc`, que compara dos personas `p1` y `p2`, determinando que `p1` precede a `p2` si su nombre es alfabéticamente

menor; `cmpPersonasNomDesc`, que indicará que *p1* precede a *p2* si su nombre es alfabéticamente mayor; y `cmpPersonasDNI`, que determina que *p1* precede a *p2* si su número de documento es inferior.

```
int cmpPersonasNomAsc(Persona p1, Persona p2)
{
    return p1.nom<p2.nom?-1:p1.nom>p2.nom?1:0;
}

int cmpPersonasNomDesc(Persona p1, Persona p2)
{
    return p2.nom<p1.nom?-1:p2.nom>p1.nom?1:0;
}

int cmpPersonasDNI(Persona p1, Persona p2)
{
    return p1.dni-p2.dni;
}
```

Notemos la diferencia entre estas funciones y las anteriores. Por ejemplo: en `cmpPersonasDNI` comparamos dos personas por su número de DNI; en cambio, en `cmpPersonaDNI` la comparación se realiza entre una persona y un número de DNI.

```
// recibe dos personas
int cmpPersonasDNI(
    Persona p1
    , Persona p2)
{
    return p1.dni-p2.dni;
}
```

```
// recibe persona e int
int cmpPersonaDNI(
    Persona p
    , int dni)
{
    return p.dni-dni;
}
```

Ahora sí, podemos usar una colección de personas en nuestro programa.

```
int main()
{
    // creo una coleccion de personas
    Coll<Persona> c = coll<Persona>();
}
```

```

// le agrego tres personas
Persona p = {10, "Pedro"};
collAdd<Persona>(c,p,personaToString);

p = {20, "Pablo"};
collAdd<Persona>(c,p,personaToString);

p = {30, "Juan"};
collAdd<Persona>(c,p,personaToString);

// itero la coleccion
collReset<Persona>(c);
while(collHasNext<Persona>(c))
{
    // obtengo siguiente persona
    Persona p = collNext<Persona>(c,personaFromString);
    cout<< v.dni << "," << v.nombre << endl;
}

// el usuario ingresa un numero de DNI
int dni;
cout << "Ingrese un dni: ";
cin >> dni;

// busco una persona x DNI...
int pos=collFind<Persona,int>(c
                                ,dni
                                ,cmpPersonaDNI
                                ,personaFromString);

// si existe una persona con el DNI ingresado...
if( pos>=0 )
{
    // obtengo la persona y la muestro
    Persona p=collGetAt<Persona>(c
                                ,pos
                                ,personaFromString);

    cout << p.dni << "," << p.nombre << endl;
}
else
{

```

```

        // no existe una persona con ese DNI
        cout << "Persona no encontrada" << endl;
    }

    // ordeno la coleccion por nombre ascendente
    collSort<Persona>(c
                    , cmpPersonasNomAsc
                    , personaFromString
                    , personaToString);

    // muestro la coleccion ordenada
    collReset<Persona>(c);
    while(collHasNext<Persona>(c))
    {
        Persona p = collNext<Persona>(c, personaFromString);
        cout << v.dni << "," << v.nombre << endl;
    }

    // ordeno la coleccion por nombre descendente
    collSort<Persona>(c
                    , cmpPersonasNomDesc
                    , personaFromString
                    , personaToString);

    // muestro la coleccion ordenada por nombre descendente
    collReset<Persona>(c);
    while(collHasNext<Persona>(c))
    {
        Persona p = collNext<Persona>(c, personaFromString);
        cout << v.dni << "," << v.nombre << endl;
    }

    // ordeno la coleccion por DNI
    collSort<Persona>(c
                    , cmpPersonasDNI
                    , personaFromString
                    , personaToString);

    // muestro la coleccion ordenada por DNI
    collReset<Persona>(c);
    while(collHasNext<Persona>(c))
    {
        Persona p = collNext<Persona>(c, personaFromString);
        cout << v.dni << "," << v.nombre << endl;
    }

```

```

    return 0;
}

```

2.6.1.4. Colección de colecciones

El TAD `Coll` permite mantener en memoria una colección de objetos tipo `T`. Dado que `T` es un tipo genérico, perfectamente podría tratarse de una colección en sí mismo. De este modo, cada uno de los elementos de nuestra colección será una colección de elementos; y la colección principal, una colección de colecciones.

Si declaramos: `Coll<Coll<int> > c`, la variable `c` representará un conjunto de datos como el que ilustra la siguiente imagen.

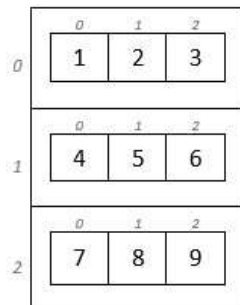


Figura 2.4. Colección de colecciones

En la imagen vemos que la colección principal tiene tres elementos, cada uno de los cuales es una colección que contiene tres elementos tipo `int`.

Para usar esta colección de colecciones debemos disponer de las funciones de conversión `tToString` y `tFromString`, que en este caso serán: `collIntToString` y `collIntFromString`.

```

string collIntToString(Coll<int> x)
{
    string r = "";
    collReset<int>(x);
    while( collHasNext<int>(x) )
    {
        int i = collNext<int>(x, stringToInt);
        addToken(r, ',', intToString(i));
    }
}

```

```

    }

    return r;
}

```

Observemos que la cadena que construimos en `collIntToString` utiliza el carácter `' '` para separar entre sí los elementos de la colección que representa.

Si la colección `c`, que declaramos e ilustramos previamente, tuviese las colecciones `{1,2,3}`, `{4,5,6}` y `{7,8,9}` (tal cual la figura), su cadena *tokenizada* será:

`"1,2,3|4,5,6|7,8,9"`

Esto quiere decir que una colección de colecciones encapsula una cadena *tokenizada*, donde cada *token* es, en sí mismo, una cadena *tokenizada*.

Veamos la función inversa, que recibe una cadena y retorna una colección.

```

Coll<int> collIntFromString(string s)
{
    Coll<int> x = coll<int>(' ');
    for(int i=0; i<tokenCount(s, '|'); i++)
    {
        int i = stringToInt(getTokenAt(s, '|', i));
        collAdd<int>(x, i, intToString);
    }

    return x;
}

```

Ahora sí, estamos en condiciones de usar una colección de colecciones en nuestro programa, tal como vemos en el siguiente ejemplo.

```

int main()
{
    Coll<Coll<int> > c = coll<Coll<int> >();
}

```

```

// debemos especificar separador diferente a '|'
Coll<int> x = coll<int>(' ');
collAdd<int>(x,1,intToString);
collAdd<int>(x,2,intToString);
collAdd<int>(x,3,intToString);

// la agrego a la coleccion de colecciones
collAdd<Coll<int> >(c,x,collIntToString);

// otra coleccion
x = coll<int>(' ');
collAdd<int>(x,4,intToString);
collAdd<int>(x,5,intToString);
collAdd<int>(x,6,intToString);

// la agrego a la coleccion de colecciones
collAdd<Coll<int> >(c,x,collIntToString);

// otra coleccion mas...
x = coll<int>(' ');
collAdd<int>(x,7,intToString);
collAdd<int>(x,8,intToString);
collAdd<int>(x,9,intToString);

// la agrego a la coleccion de colecciones
collAdd<Coll<int> >(c,x,collIntToString);

// ahora itero la coleccion principal
while( collHasNext<Coll<int> >(c) )
{
    cout << "[";

    // obtengo el siguiente elemento (que es: Coll<int>)
    Coll<int> z=collNext<Coll<int> >(c,collIntFromString);

    collReset<int>(z);
    while( collHasNext<int>(z) )
    {
        int n = collNext<int>(z,stringToInt);
        cout << n << " ";
    }
    cout << "]" ";
}

```

```

    cout << endl;
    return 0;
}

```

La salida del programa será:

```
[1 2 3] [4 5 6] [7 8 9]
```

2.6.1.5. Colecciones de estructuras que tienen colecciones

Perfectamente, alguno de los campos de una estructura podría ser una colección. Como es el caso de la estructura `Conjunto`, cuyo código vemos a continuación.

```

struct Conjunto
{
    string nom; // nombre del conjunto
    Coll<int> c; // elementos del conjunto
};

```

Si declaramos `Coll<Conjunto> cc`, tendremos un conjunto de datos como el que ilustra la siguiente figura.

0	nom pares	c <table><tr><td>2</td><td>4</td><td>6</td><td>8</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>	2	4	6	8	0	1	2	3		
2	4	6	8									
0	1	2	3									
1	nom impares	c <table><tr><td>1</td><td>3</td><td>5</td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	1	3	5	0	1	2				
1	3	5										
0	1	2										
2	nom primos	c <table><tr><td>2</td><td>3</td><td>5</td><td>7</td><td>11</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>	2	3	5	7	11	0	1	2	3	4
2	3	5	7	11								
0	1	2	3	4								

Figura 2.5. Colección de estructuras, cada una de las cuales tiene una colección

Usar esta estructura requiere funciones de conversión `tToString` y `tFromString`, que en este caso serán: `conjuntoToString` y `conjuntoFromString`.

```

string conjuntoToString(Conjunto conjunto)
{
    string s=conjunto.nom+"#";
    collReset<int>(conjunto.c);

    int size=collSize<int>(conjunto.c);
    for(int i=0;i<size;i++)
    {
        int v = collGetAt<int>(conjunto.c,i,stringToInt);
        s+=intToString(v)+((i<size-1)?" ":"");
    }

    return s;
}

Conjunto conjuntoFromString(string s)
{
    Conjunto conjunto;

    // nombre del conjunto
    conjunto.nom = getTokenAt(s,'#',0);

    // coleccion de elementos
    conjunto.c = coll<int>(' ','');

    // cadena tokenizada con los elementos del conjunto
    string x = getTokenAt(s,'#',1);

    int n=tokenCount(x,' ','');
    for(int i=0;i<n;i++)
    {
        // agrego los elementos a la coleccion del conjunto
        int v = stringToInt(getTokenAt(x,' ','',i));
        collAdd<int>(conjunto.c,v,intToString);
    }

    return conjunto;
}

```

Ahora, en nuestro programa podemos usar una colección de elementos tipo Conjunto, como vemos a continuación.

```

int main()
{
    // conjunto de los numeros pares
    Coll<Conjunto>conjuntos = coll<Conjunto>();
    Conjunto c1 = {"pares",coll<int>('','')};
    collAdd<int>(c1.c,2,intToString);
    collAdd<int>(c1.c,4,intToString);
    collAdd<int>(c1.c,6,intToString);
    collAdd<int>(c1.c,8,intToString);
    collAdd<Conjunto>(conjuntos,c1,conjuntoToString);

    // conjuntos de los numeros impares
    Conjunto c2 = {"impares",coll<int>('','')};
    collAdd<int>(c2.c,1,intToString);
    collAdd<int>(c2.c,3,intToString);
    collAdd<int>(c2.c,5,intToString);
    collAdd<Conjunto>(conjuntos,c2,conjuntoToString);

    // conjuntos de los numeros primos
    Conjunto c3 = {"primos",coll<int>('','')};
    collAdd<int>(c3.c,2,intToString);
    collAdd<int>(c3.c,3,intToString);
    collAdd<int>(c3.c,5,intToString);
    collAdd<int>(c3.c,7,intToString);
    collAdd<int>(c3.c,11,intToString);
    collAdd<Conjunto>(conjuntos,c3,conjuntoToString);
    collReset<Conjunto>(conjuntos);
    while( collHasNext<Conjunto>(conjuntos) )
    {
        Conjunto conj=collNext<Conjunto>(conjuntos
                                           ,conjuntoFromString);

        // nombre del conjunto
        cout << conj.nom << endl;
        // elementos del conjunto
        collReset<int>(conj.c);
        while( collHasNext<int>(conj.c) )
        {
            int v = collNext<int>(conj.c,stringToInt);
            cout << v << endl;
        }
    }
    return 0;
}

```

2.6.2. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

2.7. ¿Qué sigue?

En el próximo capítulo aprenderemos a usar archivos. De este modo los datos que procesan los programas, así como la información que generan podrá provenir desde, y/o volcarse hacia, un archivo almacenado en el disco duro de la computadora.

Cuando los datos perduran más allá del tiempo en que el programa que los consume y/o produce está en ejecución, decimos que son persistentes. Los archivos son un medio a través del cual podemos lograr dicha persistencia de datos.